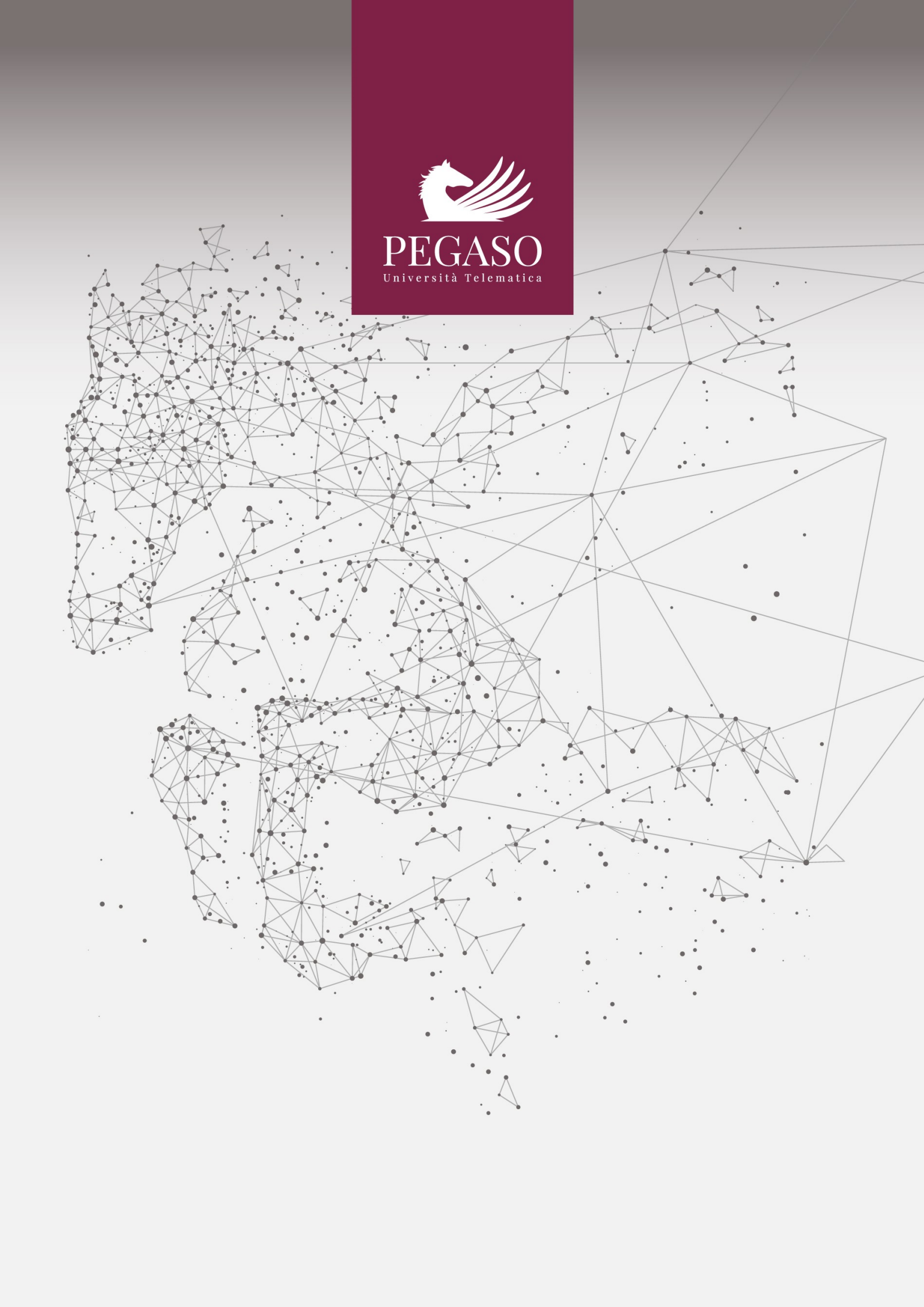

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. DDL PER LA MODIFICA....4
3. DML PER LA MODIFICA....7
4. TRIGGER....10
BIBLIOGRAFIA....13

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustrano sia i comandi per la modifica dei database, sia il concetto di trigger. In particolare, per quanto riguarda i comandi, vengono presentati sia i comandi DDL per la modifica "strutturale" delle tabelle e del database, sia i comandi DML per la modifica dei dati contenuti nelle tabelle. Infine, viene presentato il trigger in SQL, un meccanismo essenziale per intercettare gli eventi in cui si verificano specifiche condizioni sul valore dei dati delle tabelle indicando le azioni da intraprendere di conseguenza.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. DDL per la modifica

Il linguaggio DDL di SQL, oltre al comando CREATE, prevede altri tre comandi principali per la modifica dei DB:

1. ALTER;
2. RENAME;
3. DROP.

È importante comprendere che, in generale, i comandi DDL modificano la struttura del database e delle tabelle, mentre i comandi DML modificano una o più tuple all'interno di una tabella.

Il comando ALTER può essere usato per modificare una tabella aggiungendo, togliendo o modificando un campo, una chiave, un indice, ecc. La sintassi del comando ALTER è:

```
ALTER TABLE <nome_tabella> <operazione_di_modifica>;
```

Si riportano, ora, alcuni esempi di utilizzo del comando ALTER. Nel primo esempio, si vuole creare un indice sul campo relativo alla data di nascita nella tabella «Personale». Il codice SQL corrispondente è il seguente:

```
ALTER TABLE Personale
ADD INDEX indice_data_nascita(data_nascita);
```

Nel secondo esempio, si vuole creare la chiave primaria per la tabella «CronologiaPersonale» sulla combinazione dei campi relativi alla matricola e alla data di operazione. Il codice SQL corrispondente è il seguente:

```
ALTER TABLE CronologiaPersonale
ADD PRIMARY KEY(matricola,data_operazione);
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel secondo esempio, si vogliono cambiare nome e dimensione del campo matricola della tabella «Personale». Il codice SQL corrispondente è il seguente:

```
ALTER TABLE Personale
CHANGE matricola numero_matricola(6) NOT NULL;
```

Nel terzo esempio, si vuole modificare il tipo della colonna prezzo nella tabella «Prodotti». Il codice SQL corrispondente è il seguente:

```
ALTER TABLE Prodotti
MODIFY prezzo FLOAT NOT NULL;
```

Nel quarto esempio, si vuole aggiungere il campo relativo al costo di produzione nella tabella «Prodotti». Il codice SQL corrispondente è il seguente:

```
ALTER TABLE Prodotti
ADD costo_produzione DOUBLE NOT NULL DEFAULT 0;
```

Nel quinto esempio, si vuole rimuovere il campo relativo al costo di produzione dalla tabella «Prodotti». Il codice SQL corrispondente è il seguente:

```
ALTER TABLE Prodotti
DROP costo_produzione;
```

Nel sesto esempio, si vuole aggiungere la chiave esterna nella tabella «Composizione» relativa all'integrità referenziale con la tabella «Prodotti». Il codice SQL corrispondente è il seguente:

```
ALTER TABLE Composizione
ADD CONSTRAINT ProdottiComposizione FOREIGN KEY(id_prod)
REFERENCE Prodotti(id_prod);
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il comando RENAME permette la ridenominazione di una tabella esistente. La sintassi del comando RENAME è:

RENAME TABLE <vecchio_nome> TO <nuovo_nome>;

Il comando DROP, invece, permette l'eliminazione di un qualsiasi oggetto del database. La sintassi del comando DROP è:

DROP <nome_oggetto>;

Si riportano ora alcuni esempi di utilizzo del comando DROP. Nel primo esempio si vuole eliminare il database «test». Il codice SQL corrispondente è il seguente:

DROP DATABASE test;

Nel secondo esempio si vuole eliminare la tabella "CronologiaPersonale". Il codice SQL corrispondente è il seguente:

DROP TABLE CronologiaPersonale;

Nel terzo esempio si vuole eliminare l'indice indice_data_nascita dalla tabella «Personale». Il codice SQL corrispondente è il seguente:

DROP INDEX indice_data_nascita ON Personale;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. DML per la modifica

Il linguaggio DML di SQL prevede alcuni comandi fondamentali per inserire, cancellare e modificare i dati nelle tabelle di un database:

1. INSERT;
2. DELETE;
3. UPDATE.

Per il comando INSERT sono previste due forme generali:

```
INSERT INTO <tabella> [ (<lista_campi>) ]
VALUES (<lista valori>);
```

oppure

```
INSERT INTO <tabella> SET <campo_1> = <valore_1>, … ,
                          <campo_n> = <valore_n>;
```

Si riporta un esempio di utilizzo del comando INSERT riportato per entrambe le forme generali. Si vogliono inserire i dati relativi al nuovo dipartimento di Firenze. Nella prima forma generale si ha:

```
INSERT INTO Dipartimenti
            (id_dip,nome_dipartimento,localita, provincia)
VALUES ('D7','Dipartimento 7','Firenze','FI');
```

Nella seconda forma generale si ha:

```
INSERT INTO Dipartimenti
    SET id_dip='D7',nome_dipartimento = 'Dipartimento 7',
    localita = 'Firenze', provincia = 'FI';
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Come è possibile osservare nella prima forma generale, il comando INSERT consente di omettere l'elenco dei campi. Se si omette l'elenco dei campi devono però essere specificati i valori per ogni campo della tabella nell'ordine in cui questi sono stati definiti nella creazione della tabella stessa.

Se non si omette l'elenco dei campi, invece, è possibile utilizzare un ordine diverso da quello stabilito al momento della creazione della tabella ed omettere alcuni campi che verranno impostati a NULL. In quest'ultimo caso, il comando verrà rifiutato dal DBMS nel caso in cui siano specificati i campi che fanno parte della chiave primaria della tabella o campi per i quali in fase di creazione della tabella stessa sia stato specificato il vincolo NOT NULL

Come è possibile osservare nella seconda forma generale, il comando INSERT consente di utilizzare un ordine diverso dei campi rispetto a quello stabilito al momento della creazione della tabella e omettere alcuni campi che verranno impostati a NULL. In quest'ultimo caso il comando verrà rifiutato dal DBMS nel caso in cui siano specificati i campi che fanno parte della chiave primaria della tabella o campi per i quali in fase di creazione della tabella stessa sia stato specificato il vincolo NOT NULL.

Il linguaggio SQL permette anche di inserire dati in una tabella estrapolandoli dal risultato di una query. Ad esempio, si vuole scrivere la query che permette di inserire nella tabella «Componenti» un insieme di nuovi componenti estratti dalla tabella «NuoviComponenti». Il codice SQL che permette di ottenere ciò è:

```
INSERT INTO Componenti id_comp,nome_componente,fornitore)
    SELECT id_comp,nome_componente,fornitore
    FROM NuoviComponenti;
```

Per cancellare i dati di una tabella, il linguaggio SQL prevede il comando DELETE la cui sintassi è:

```
DELETE FROM <tabella> [WHERE <condizione>];
```

Se nel comando DELETE non viene specificata la clausola WHERE, l'operazione si applica a tutte le righe della tabella. Si riporta, ora, un esempio di utilizzo del comando DELETE in cui si vogliono eliminare i dati dei componenti acquistati dal fornitore Rossi all'interno della tabella «Componenti». Il codice SQL corrispondente è:

```
DELETE FROM Componenti WHERE fornitore='Rossi';
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il comando UPDATE permette di aggiornare i dati memorizzati in una tabella. La sintassi del comando UPDATE è:

```
UPDATE <tabella> SET <campo_1> = <valore_1>,…,
                     <campo_n> = <valore_n>
[WHERE <condizione>];
```

Se nel comando UPDATE non viene specificata la clausola WHERE, l'operazione si applica a tutte le righe della tabella.

Si riportano due esempi di utilizzo del comando UPDATE. Nel primo esempio si vuole aggiornare lo stipendio (di una percentuale pari a +10%) con matricola 00AC:

```
UPDATE Personale
SET stipendio=stipendio*1.1
WHERE matricola='00AC';
```

Per il secondo esempio, si supponga di aver aggiunto alla tabella «Dipartimenti» un nuovo campo N_dipendenti per contenere il numero di dipendenti di ogni singolo dipartimento e di voler aggiornare in modo automatico tale valore. Il comando che segue risolve il problema mediante una sola UPDATE che utilizza una subquery e una join:

```
UPDATE Dipartimenti, (SELECT id_dip, COUNT(*) AS N_dipendenti
                      FROM Personale
                      GROUP BY id_dip) AS T
SET Dipartimenti.N_dipendenti = T.N_dipendenti
WHERE Dipartimenti.id_dip = T.id_dip;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Trigger

In SQL un trigger è un meccanismo per intercettare gli eventi in cui si verificano specifiche condizioni sul valore dei dati indicando le azioni da intraprendere di conseguenza. Gli eventi per i quali si attiva un trigger corrispondono all'esecuzione di un'istruzione INSERT, UPDATE o DELETE su una tabella.
I trigger sono utilizzati per diversi scopi nella progettazione e nell'implementazione di un database:

1. per mantenere l'integrità referenziale tra tabelle;
2. per mantenere l'integrità dei dati di una singola tabella;
3. per verificare il contenuto dei campi di una tabella;
4. per creare tabelle di registrazione delle modifiche o delle eliminazioni apportate al database.

La sintassi di un trigger è la seguente:

```
CREATE TRIGGER <nome_trigger>
{ BEFORE | AFTER }
{ INSERT  | UPDATE  | DELETE }
ON <nome_tabella>
FOR EACH ROW
<codice_SQL>
```

dove:

- FOR EACH ROW permette di fare riferimento alle righe della tabella interessate dall'evento;
- NEW fa riferimento al nuovo valore dei campi delle righe interessate dall'evento;
- OLD fa riferimento al vecchio valore dei campi delle righe interessate dall'evento.

Si riportano tre esempi significativi di utilizzo del trigger in SQL. Nel primo, il trigger deve attivare, quando il costo di un componente viene aggiornato, l'inserimento dei dati di tale componente in una tabella (di log) chiamata «CronologiaCostiComponenti» per registrare l'avvenuto aggiornamento.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice SQL del trigger è il seguente:

```
CREATE TRIGGER Aggiorna_Costo_Componente
AFTER UPDATE
ON Componenti
FOR EACH ROW
BEGIN
   IF NEW.costo_unitario <> OLD.costo_unitario
    INSERT INTO CronologiaCostiComponenti
        VALUES(OLD.id_comp,OLD.nome_componente,
        OLD.costo_unitario, CURRENT_DATE());
  END IF;
END;
```

Nel secondo esempio, il trigger deve attivare, quando viene cancellata una riga della tabella «Personale», l'inserimento dei dati dell'impiegato cancellato in un'opportuna tabella chiamata «CronologiaPersonale» che prevede la registrazione della matricola, del codice di dipartimento, del nominativo, del tipo di operazione e della data dell'operazione. Il codice SQL del trigger è il seguente:

```
CREATE TRIGGER Elimina_impiegato
BEFORE DELETE
ON Personale
FOR EACH ROW
INSERT INTO CronologiaPersonale
VALUES(OLD.matricola,OLD.id_dip,OLD.nominativo,'Eliminato',
    CURRENT_DATE());
```

Nel terzo esempio, il trigger deve attivare, quando viene inserito un nuovo prodotto, il controllo sul prezzo di vendita ponendo il prezzo uguale a zero se è stato inserito un valore negativo oppure ponendo il prezzo uguale a 250 se è stato inserito un valore maggiore di 250.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice SQL del trigger è il seguente:

```
CREATE TRIGGER Nuovo_Prodotto
BEFORE INSERT
ON Prodotti
FOR EACH ROW
BEGIN
    IF NEW.prezzo < 0 THEN
       SET NEW.prezzo = 0;
    END IF;
   IF NEW.prezzo > 250.00 THEN
       SET NEW.prezzo = 250.00;
   END IF;
END;
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Formichi, F., Meini, G. (2018). Corso di Informatica (seconda edizione) Vol. 3., Zanichelli.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).